0.1 Lý thuyết số và đại số trừu tượng

0.1.1 Số nguyên tố và phép đồng dư

0.1.1.1 Số nguyên tố và định lý cơ bản của số học

Định nghĩa 0.1 (Số nguyên tố)**.** Số nguyên tố là số nguyên dương $p > 1$ chỉ có đúng hai ước dương là 1 và chính nó. Tập các số nguyên tố ký hiệu là $\mathbb{P}$.

Định lý 0.2 (Euclid)**.** *Tập các số nguyên tố là vô hạn.*

Định lý 0.3 (Định lý cơ bản của số học)**.** *Mọi số nguyên $n > 1$ đều có duy nhất một cách viết dưới dạng*

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r},$$

trong đó $p_1 < p_2 < \cdots < p_r$ là các số nguyên tố và $e_i \in \mathbb{N}^*$.

Trong mật mã, ta đặc biệt quan tâm đến các số nguyên tố *lớn* — thường có độ dài từ 2048 bit trở lên (nếu là $2048 bit th 2^{2047} \leq n < 2^{2048}$) — vì chúng tạo nên nền tảng cho các bài toán khó như phân tích thừa số nguyên (RSA) và logarit rời rạc (ElGamal, DSA, ECDSA).

0.1.1.2 Phép chia có dư và phép đồng dư

Định lý 0.4 (Phép chia có dư)**.** *Với mọi $a \in \mathbb{Z}$ và $n \in \mathbb{Z}^+$, tồn tại duy nhất cặp số nguyên (q, r) sao cho*

$$a = qn + r, \quad 0 \leq r < n.$$

Ta gọi q là thương*, r là* số dư*, ký hiệu $r = a \bmod n$.*

Định nghĩa 0.5 (Đồng dư)**.** Hai số nguyên a, b được gọi là *đồng dư* modulo n, ký hiệu $a \equiv b \pmod{n}$, nếu $n \mid (a - b)$, tức a và b có cùng số dư khi chia cho n.

Mệnh đề 0.6 (Tính chất cơ bản của đồng dư)**.** *Phép đồng dư là một quan hệ tương đương trên $\mathbb{Z}$ và tương thích với các phép toán cộng, trừ, nhân. Cụ thể, nếu $a \equiv b \pmod{n}$ và $c \equiv d \pmod{n}$ thì:*

1. $a \pm c \equiv b \pm d \pmod{n}$,
2. $ac \equiv bd \pmod{n}$,
3. $a^k \equiv b^k \pmod{n}$ *với mọi* $k \in \mathbb{N}$.

Tính chất này cho phép thực hiện toàn bộ các phép tính của ElGamal trong tập hữu hạn $\{0, 1, \ldots, p - 1\}$ — không bao giờ bị tràn số dù p có cỡ 2048 bit hay lớn hơn.

Tập các lớp tương đương modulo n cùng phép cộng và phép nhân tạo thành *vành thương* $\mathbb{Z}/n\mathbb{Z}$, ký hiệu tắt là $\mathbb{Z}_n$. Tập các phần tử khả nghịch (đối với phép nhân) của $\mathbb{Z}_n$ ký hiệu là $\mathbb{Z}_n^*$.

0.1.1.3 Định lý Fermat nhỏ và định lý Euler

Định lý 0.7 (Fermat nhỏ)**.** *Cho p là số nguyên tố và $a \in \mathbb{Z}$ với $\gcd(a, p) = 1$. Khi đó*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Hệ quả: với mọi $a \in \mathbb{Z}$, ta có $a^p \equiv a \pmod{p}$.

Định nghĩa 0.8 (Hàm Euler φ)**.** Hàm Euler $\varphi : \mathbb{Z}^+ \to \mathbb{Z}^+$ đếm số các số nguyên trong $\{1, 2, \ldots, n\}$ nguyên tố cùng nhau với n:

$$\varphi(n) = |\{a : 1 \leq a \leq n,\ \gcd(a, n) = 1\}|.$$

Đặc biệt, $\varphi(p) = p - 1$ khi p là số nguyên tố.

Định lý 0.9 (Euler)**.** *Với mọi $n \in \mathbb{Z}^+$ và $a \in \mathbb{Z}$ thỏa $\gcd(a, n) = 1$:*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Định lý Fermat nhỏ đóng vai trò then chốt trong bộ công cụ mật mã:

- Cho phép tính nghịch đảo nhanh *khi modulo là số nguyên tố*: $a^{-1} \equiv a^{p-2} \pmod{p}$.
- Là cơ sở cho các thuật toán kiểm tra tính nguyên tố xác suất như Fermat test, Miller–Rabin — được dùng đ sinh các số nguyên tố lớn trong giai đoạn KeyGen của ElGamal.
- Là nền tảng để chứng minh tính đúng đắn của lược đồ chữ ký ElGamal: lũy thừa trên $\mathbb{Z}_p^*$ chỉ phụ thuộc vào số mũ modulo $p - 1$.

0.1.1.4 Nghịch đảo modular và thuật toán Euclid mở rộng

Định nghĩa 0.10 (Nghịch đảo modular)**.** Nghịch đảo modular của a theo modulo n là số nguyên a^{-1} thỏa mãn $a \cdot a^{-1} \equiv 1 \pmod{n}$.

Định lý 0.11. *Nghịch đảo modular của a modulo n tồn tại khi và chỉ khi $\gcd(a, n) = 1$.*

Thuật toán Euclid mở rộng (Extended Euclidean Algorithm) tính đồng thời $\gcd(a, n)$ và các *hệ số Bézout* $(u, v) \in \mathbb{Z}^2$ thỏa mãn $au + nv = \gcd(a, n)$. Khi $\gcd(a, n) = 1$, ta có $a^{-1} \equiv u \pmod{n}$.

Thuật toán 0.1: Thuật toán Euclid mở rộng

Đầu vào: $a, n \in \mathbb{Z}^+$
Đầu ra: (d, u, v) với $d = \gcd(a, n)$ và $au + nv = d$
$(r_0, r_1) \leftarrow (a, n)$;
$(u_0, u_1) \leftarrow (1, 0)$;
$(v_0, v_1) \leftarrow (0, 1)$;
while $r_1 \neq 0$ **do**
  $q \leftarrow \lfloor r_0/r_1 \rfloor$;
  $(r_0, r_1) \leftarrow (r_1,\ r_0 - q \cdot r_1)$;
  $(u_0, u_1) \leftarrow (u_1,\ u_0 - q \cdot u_1)$;
  $(v_0, v_1) \leftarrow (v_1,\ v_0 - q \cdot v_1)$;
trả về (r_0, u_0, v_0);

Độ phức tạp: $O(\log \min(a, n))$ phép chia, tổng cộng $O(k^3)$ bit-operation với $k = \log n$. Trong Python: `pow(a, -1, n)` (Python 3.8+) đã tích hợp sẵn.

Ví dụ 0.12. Tính $7^{-1} \bmod 26$. Áp dụng thuật toán Euclid trên 26 và 7:

$$26 = 3 \cdot 7 + 5, \quad 7 = 1 \cdot 5 + 2, \quad 5 = 2 \cdot 2 + 1, \quad 2 = 2 \cdot 1 + 0.$$

Quay ngược các hệ số Bézout: $1 = 5 - 2 \cdot 2 = 3 \cdot 5 - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7$. Vậy $7^{-1} \equiv -11 \equiv 15 \pmod{26}$.

Nhận xét 0.13. Trong lược đồ ElGamal, ta cần tính $k^{-1} \bmod (p-1)$ ở bước ký. Modulo ở đây là $p - 1$ — *không nguyên tố* (vì $p - 1$ luôn chẵn). Do đó **bắt buộc** phải dùng thuật toán Euclid mở rộng, không thể dùng công thức a^{p-2} của Định lý **??**. Đây là chi tiết cài đặt dễ bị nhầm khi mới làm quen với ElGamal.

0.1.1.5 Định lý số dư Trung Hoa

Định lý 0.14 (Định lý số dư Trung Hoa — CRT)**.** *Cho $n_1, n_2, \ldots, n_k$ là các số nguyên dương đôi một nguyên tố cùng nhau. Đặt $N = n_1 n_2 \cdots n_k$. Khi đó với mọi bộ số $(a_1, \ldots, a_k)$, hệ phương trình đồng dư $x \equiv a_i \pmod{n_i}$ có nghiệm duy nhất modulo N.*

CRT cho phép phân rã các phép tính trên $\mathbb{Z}_N$ thành các phép tính song song trên các $\mathbb{Z}_{n_i}$ nhỏ hơn. Đây cũng là công cụ trong thuật toán Pohlig–Hellman tấn công DLP khi $p - 1$ có các thừa số nhỏ — điều giải thích vì sao phải chọn p là *safe prime* (xem Mục **??**).

0.1.2 Nhóm hữu hạn và phần tử nguyên thủy

0.1.2.1 Khái niệm nhóm và nhóm con

Định nghĩa 0.15 (Nhóm)**.** Một *nhóm* là cặp $(G, \cdot)$ gồm tập $G \neq \emptyset$ và phép toán hai ngôi $\cdot : G \times G \to G$ thỏa mãn:

1. **Kết hợp**: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ với mọi $a, b, c \in G$.
2. **Phần tử đơn vị**: tồn tại $e \in G$ sao cho $e \cdot a = a \cdot e = a$ với mọi $a \in G$.
3. **Phần tử nghịch đảo**: với mỗi $a \in G$, tồn tại $a^{-1} \in G$ sao cho $a \cdot a^{-1} = e$.

Nhóm $(G, \cdot)$ được gọi là *Abel* nếu $a \cdot b = b \cdot a$ với mọi $a, b \in G$. *Cấp* của nhóm, ký hiệu $|G|$, là số phần tử của G.

Định nghĩa 0.16 (Nhóm con)**.** Tập con $H \subseteq G$ được gọi là *nhóm con* của G, ký hiệu $H \leq G$, nếu H cùng phép toán hạn chế từ G là một nhóm.

Định lý 0.17 (Lagrange)**.** *Nếu H là nhóm con của nhóm hữu hạn G thì $|H|$ là ước của $|G|$.*

Trong mật mã khóa công khai dựa trên DLP, ta luôn làm việc với các nhóm *hữu hạn, Abel*.

0.1.2.2 Cấp của phần tử và nhóm cyclic

Định nghĩa 0.18 (Cấp của phần tử)**.** Cho nhóm $(G, \cdot)$ và $a \in G$. *Cấp* của a, ký hiệu $\mathrm{ord}(a)$, là số nguyên dương nhỏ nhất k sao cho $a^k = e$.

Hệ quả 0.19. *Trong nhóm hữu hạn,* $\mathrm{ord}(a) \mid |G|$ *với mọi* $a \in G$.

Chứng minh. Tập $\langle a \rangle = \{a^0, a^1, \ldots, a^{\mathrm{ord}(a)-1}\}$ là nhóm con của G cấp $\mathrm{ord}(a)$. Áp dụng Định lý **??**. □

Định nghĩa 0.20 (Nhóm cyclic)**.** Nhóm G được gọi là *cyclic* nếu tồn tại phần tử $g \in G$ sao cho mọi phần tử của G đều có dạng g^k với $k \in \mathbb{Z}$. Phần tử g được gọi là *phần tử sinh* (generator), ta viết $G = \langle g \rangle$.

Định lý 0.21 (Cấu trúc nhóm cyclic)**.** *Mọi nhóm cyclic hữu hạn cấp n đều đẳng cấu với $(\mathbb{Z}_n, +)$. Với mỗi ước d của n, nhóm cyclic cấp n có đúng* một *nhóm con cấp d.*

0.1.2.3 Nhóm nhân $\mathbb{Z}_p^*$

Cho p là số nguyên tố. Tập $\mathbb{Z}_p^* = \{1, 2, \ldots, p-1\}$ cùng với phép nhân modulo p tạo thành một nhóm — đây chính là cấu trúc đại số mà ElGamal hoạt động trên đó.

Định lý 0.22 (Gauss)**.** *Với p nguyên tố, $\mathbb{Z}_p^*$ là nhóm cyclic cấp $p-1$.*

Hệ quả: tồn tại ít nhất một phần tử $g \in \mathbb{Z}_p^*$ sao cho $\mathbb{Z}_p^* = \{g^0, g^1, \ldots, g^{p-2}\}$.

0.1.2.4 Phần tử nguyên thủy

Định nghĩa 0.23 (Phần tử nguyên thủy)**.** Phần tử $g \in \mathbb{Z}_p^*$ được gọi là *phần tử nguyên thủy* (primitive root, generator) modulo p nếu $\mathrm{ord}(g) = p-1$, tức g sinh ra toàn bộ $\mathbb{Z}_p^*$.

Định lý 0.24. *Số phần tử nguyên thủy modulo p bằng $\varphi(p-1)$.*

Cách kiểm tra phần tử nguyên thủy. Cho $p-1 = q_1^{e_1} \cdots q_r^{e_r}$. Phần tử g là nguyên thủy modulo p khi và chỉ khi $g^{(p-1)/q_i} \not\equiv 1 \pmod{p}$ với mọi $i = 1, \ldots, r$.

Ví dụ 0.25 (Phần tử nguyên thủy modulo 11)**.** Với $p = 11$, $p-1 = 10 = 2 \cdot 5$. Kiểm tra $g = 2$: $2^5 = 32 \equiv 10 \not\equiv 1 \pmod{11}$ và $2^2 = 4 \not\equiv 1 \pmod{11}$. Vậy $g = 2$ là phần tử nguyên thủy. Các lũy thừa:

i	1	2	3	4	5	6	7	8	9	10
$2^i \bmod 11$	2	4	8	5	10	9	7	3	6	1

Sinh đủ 10 phần tử của $\mathbb{Z}_{11}^*$. Số phần tử nguyên thủy modulo 11 là $\varphi(10) = 4$, cụ thể là $\{2, 6, 7, 8\}$.

0.1.2.5 Safe prime

Định nghĩa 0.26 (Safe prime)**.** Số nguyên tố p được gọi là *safe prime* nếu $q = (p-1)/2$ cũng là số nguyên tố. Khi đó q được gọi là số nguyên tố Sophie Germain.

Trong thực hành mật mã, ta luôn chọn p là safe prime vì ba lý do:

1. **Kiểm tra phần tử nguyên thủy cực nhanh.** $p-1 = 2q$ chỉ có hai thừa số nguyên tố, chỉ cần kiểm tra $g^{(p-1)/2} \neq 1$ và $g^2 \neq 1$.
2. **Hầu như mọi phần tử đều có cấp lớn.** Cấp của phần tử là ước của $2q$, chỉ có thể là $1, 2, q, 2q$. Chỉ $g = 1$ có cấp 1 và $g = p-1$ có cấp 2; mọi phần tử còn lại có cấp q hoặc $2q$.
3. **Chống tấn công Pohlig–Hellman.** Tấn công này khai thác các thừa số nhỏ của $p-1$ để giải DLP nhanh. Khi $p-1 = 2q$ với q nguyên tố lớn, không tồn tại nhóm con nhỏ để khai thác.

0.1.3 Lũy thừa modular

Bài toán tính $g^x \bmod p$ với g, x, p lớn (cỡ vài nghìn bit) là phép toán cơ bản nhất trong mọi hệ mã khóa công khai dựa trên DLP, xuất hiện ở cả ba bước KeyGen, Sign và Verify của ElGamal.

0.1.3.1 Tại sao cần thuật toán hiệu quả

Tính trực tiếp g^x rồi mới rút gọn modulo p là *không khả thi*: với g, x cỡ 2048 bit, giá trị g^x có thể chứa tới $\sim 10^{619}$ chữ số — vượt xa khả năng lưu trữ của mọi máy tính. Ý tưởng là rút gọn modulo p *sau mỗi phép nhân* và dùng biểu diễn nhị phân của số mũ để giảm số phép nhân từ x xuống $O(\log x)$.

0.1.3.2 Thuật toán bình phương và nhân

Ý tưởng. Biểu diễn $x = (b_{n-1} \dots b_1 b_0)_2$. Khi đó $g^x = \prod_{i:\, b_i = 1} g^{2^i}$, và các g^{2^i} được tính bằng bình phương liên tiếp, mỗi bước rút gọn modulo p.

Thuật toán 0.2: Lũy thừa modular — bình phương và nhân

Đầu vào: g, x, p với $x \geq 0$, $p > 0$
Đầu ra: $g^x \bmod p$

```
result ← 1;
base ← g mod p;
while x > 0 do
    if x lẻ then
        result ← result · base mod p;
    x ← ⌊x/2⌋;
    base ← base^2 mod p;
trả về result;
```

Phân tích độ phức tạp. Số vòng lặp đúng bằng $\lfloor \log_2 x \rfloor + 1$, tổng số phép nhân không vượt $2 \log_2 x$. Với x cỡ 2048 bit, chỉ cần khoảng 4000 phép nhân — hoàn toàn thực hiện được trong vài mili giây. Tổng độ phức tạp $O(\log^3 p)$ bit-operation. Trong Python: `pow(g, x, p)` đã cài đặt thuật toán này.

Ví dụ 0.27. Tính $3^{13} \bmod 17$. Ta có $13 = (1101)_2$, do đó $3^{13} = 3^8 \cdot 3^4 \cdot 3^1$.

$$3^1 = 3, \quad 3^2 = 9, \quad 3^4 \equiv 13 \pmod{17}, \quad 3^8 \equiv 16 \pmod{17}.$$

Suy ra $3^{13} \equiv 16 \cdot 13 \cdot 3 = 624 \equiv 12 \pmod{17}$.

0.1.3.3 *Tính bất đối xứng — nền tảng cho mật mã khóa công khai*

Chiều thuận	$x \longmapsto y = g^x \bmod p$ là *dễ*: $O(\log^3 p)$ bit-op.
Chiều ngược	$(g, y, p) \longmapsto x$ (DLP) là *khó*: dưới hàm mũ.

Tính bất đối xứng này chính là tính chất *cửa sập một chiều* (one-way trapdoor) — nền tảng của mật mã khóa công khai do Diffie và Hellman đề xuất năm 1976. Mục **??** tiếp theo sẽ đi sâu vào bài toán logarit rời rạc.

0.2 Bài toán Logarit rời rạc

Mục này phát biểu bài toán logarit rời rạc (Discrete Logarithm Problem — DLP), thảo luận tại sao DLP được tin tưởng là khó về mặt tính toán, và liên hệ với lý thuyết độ phức tạp.

0.2.1 Phát biểu bài toán

0.2.1.1 *Logarit thông thường và động cơ*

Trong giải tích thực, phép lấy logarit $\log_g(\cdot)$ là dễ. Khi chuyển sang nhóm hữu hạn cyclic, định nghĩa logarit vẫn được giữ nguyên về mặt cấu trúc, nhưng tính *dễ tính* thì biến mất hoàn toàn — đây chính là điểm mấu chốt mà mật mã khai thác.

0.2.1.2 *Phát biểu trong nhóm cyclic*

Định nghĩa 0.28 (Bài toán logarit rời rạc — DLP)**.** Cho $(G, \cdot)$ là nhóm cyclic hữu hạn cấp n, sinh bởi g. Cho $y \in G$. Tìm $x \in \{0, 1, \ldots, n-1\}$ sao cho $g^x = y$. Số x này ký hiệu là $x = \log_g y$.

Mệnh đề 0.29 (Tính tồn tại và duy nhất)**.** *Vì $G = \langle g \rangle$ cyclic cấp n, mọi $y \in G$ có biểu diễn duy nhất g^x với $x \in \{0, 1, \ldots, n-1\}$. Do đó logarit rời rạc luôn tồn tại và duy nhất.*

0.2.1.3 *Phát biểu cụ thể trên $\mathbb{Z}_p^*$*

> **DLP trên $\mathbb{Z}_p^*$.** Cho số nguyên tố p, phần tử nguyên thủy $g \in \mathbb{Z}_p^*$, và $y \in \mathbb{Z}_p^*$. Tìm $x \in \{0, 1, \ldots, p-2\}$ sao cho $g^x \equiv y \pmod p$.

Đây chính là dạng DLP mà lược đồ chữ ký ElGamal dựa vào.

Ví dụ 0.30 (DLP với tham số nhỏ)**.** Với $p = 11, g = 2$ (phần tử nguyên thủy theo Ví dụ **??**), cho $y = 7$. Tra bảng lũy thừa: $2^7 \equiv 7 \pmod{11}$, nên $\log_2 7 = 7$ trong $\mathbb{Z}_{11}^*$. Khi $p \approx 2^{2048}$, không gian tìm kiếm lên tới 2^{2048} khả năng — vượt xa mọi máy tính cổ điển.

0.2.1.4 *Tính chất một chiều*

- Tính theo chiều thuận $x \mapsto g^x \bmod p$ là *dễ*: $O(\log^3 p)$ (Mục **??**).
- Tính theo chiều ngược $y \mapsto \log_g y$ là *khó*: chưa có thuật toán đa thức nào được biết.

0.2.2 Tại sao DLP khó

0.2.2.1 *Không có thuật toán đa thức được biết*

Tính đến nay, **không có thuật toán đa thức nào được biết để giải DLP trên $\mathbb{Z}_p^*$ tổng quát**. Các thuật toán tốt nhất:

- *Tổng quát* (mọi nhóm cyclic): $O(\sqrt{p})$ — Baby-step Giant-step, Pollard rho.
- *Riêng cho* $\mathbb{Z}_p^*$ (khai thác cấu trúc số học): dưới hàm mũ $L_p[1/3, c]$ — Index Calculus, Number Field Sieve.

Niềm tin vào độ khó của DLP đến từ lịch sử nghiên cứu hơn 50 năm chưa có đột phá, sự chứng nhận của NSA/NIST, và tính phổ quát của bài toán trong nhiều lĩnh vực toán học.

0.2.2.2 *Liên hệ với lý thuyết độ phức tạp*

Phiên bản quyết định DLP-DECISION (cho (p, g, y, k), có $x \le k$ sao cho $g^x \equiv y$?) nằm trong $\mathbf{NP} \cap \mathbf{coNP}$.

Mệnh đề 0.31. DLP-DECISION $\in \mathbf{NP} \cap \mathbf{coNP}$.

Phác thảo. Thuộc NP*:* Chứng nhận "có" là giá trị x — kiểm tra $g^x \equiv y \pmod p$ và $x \le k$ trong thời gian đa thức.

Thuộc coNP*:* Chứng nhận "không" là $x_0 = \log_g y$ thực, với $x_0 > k$ — kiểm tra tương tự. □

Nhận xét 0.32 (Vị trí của DLP trong lý thuyết phức tạp). DLP *chưa được chứng minh* thuộc $\mathbf{P}$ và cũng *chưa được chứng minh* là $\mathbf{NP}$-khó (vì mọi bài toán $\mathbf{NP}$-khó trong $\mathbf{NP} \cap \mathbf{coNP}$ sẽ kéo theo $\mathbf{NP} = \mathbf{coNP}$). *Độ khó của DLP là một giả thiết, không phải định lý* — đây là bản chất của mọi hệ mã khóa công khai.

0.2.2.3 *Liên hệ với các bài toán mật mã liên quan*

DLP. Cho (g, y), tìm $x = \log_g y$. Bài toán "khó nhất" trong họ.

CDH (Computational Diffie–Hellman). Cho (g, g^a, g^b), tính g^{ab}.

DDH (Decisional Diffie–Hellman). Cho (g, g^a, g^b, z), quyết định xem $z = g^{ab}$ hay z ngẫu nhiên.

Ta có chuỗi rút gọn: Giải DLP $\implies$ Giải CDH $\implies$ Giải DDH. Lược đồ chữ ký ElGamal dựa trên DLP; trao đổi khóa Diffie–Hellman dựa trên CDH.

0.2.2.4 Ảnh hưởng của kích thước tham số

Bit của p	**Đánh giá an toàn**	**Ghi chú**
512	Đã phá	Chỉ dùng demo
1024	Không khuyến nghị	NIST deprecated từ 2013
2048	Ngắn–trung hạn (~ 2030)	Mức tối thiểu hiện hành
3072	Dài hạn	Khuyến nghị mới của NIST
7680	Tương đương AES-256	Hiếm dùng do tốc độ chậm

0.2.2.5 Đe dọa từ máy tính lượng tử

Năm 1994, Peter Shor công bố thuật toán lượng tử giải DLP trong thời gian đa thức $O(\log^3 p)$. Các máy tính lượng tử thực tế hiện tại chưa đủ lớn để đe dọa, nhưng đây là động lực cho nghiên cứu *mật mã hậu lượng tử* (post-quantum cryptography) dựa trên các bài toán khó mới như lattice problems, code-based, hash-based.

0.2.3 Các thuật toán tấn công DLP đã biết

[TODO: Giới thiệu các thuật toán: Baby-step Giant-step ($O(\sqrt{p})$), Pohlig–Hellman, Index Calculus. Nêu độ phức tạp và điều kiện áp dụng từng thuật toán.]

0.3 Tổng quan về chữ ký số

0.3.1 Mục tiêu bảo mật

[TODO: Trình bày ba tính chất cần đảm bảo: tính xác thực (authenticity), tính toàn vẹn (integrity), tính chống chối bỏ (non-repudiation). Phân biệt với mã hóa thông thường.]

0.3.2 Mô hình tổng quát

[TODO: Mô tả ba thành phần: thuật toán sinh khóa (KeyGen), thuật toán ký (Sign), thuật toán xác minh (Verify). Vẽ sơ đồ luồng ký và xác minh giữa người ký và người nhận.]

0.3.3 Bối cảnh lịch sử

[TODO: Lịch sử phát triển chữ ký số: Diffie–Hellman (1976), RSA (1978), ElGamal (1985), DSA (1994), các chuẩn hiện đại. Vị trí của ElGamal trong lịch sử phát triển mật mã khóa công khai.]